Quetta Agents MCP

사용자 매뉴얼 & 기술 개요

Smart LLM Gateway + 원격 PC 제어 + 문서·설계도 분석 + RAG 지식베이스

Version 0.14.0

Documentation Build

https://github.com/choyunsung/quetta-agents-mcp

목차

Quetta Agents MCP — 상세 문서
01. 아키텍처 개요
02. LLM 라우팅
03. 스마트 디스패처 (`quetta_auto`)
04. 원격 에이전트 (Remote Agent)
05. 논문 분석 (Nougat + Gemini + Claude + RAG)
06. 설계도 분석 (기계 / 전기 / CPLD)
07. 파일 업로드 & RAG 통합
08. 도구 레퍼런스
09. 설치 & 설정
10. 트러블슈팅
11. 공유 메모리 & 멀티 계정
12. 비공개 설치 시스템 (2가지 방식)
13. 대화 히스토리 (NoSQL)
14. 워크스페이스 (멀티 테넌트)

Quetta Agents MCP — 상세 문서

Quetta Agents MCP는 **LLM 스마트 라우팅 + 원격 PC(GPU) 제어 + 문서/도면 분석 + RAG 지식베이스**를 하나의 MCP로 통합한 시스템입니다.

📚 문서 목차

1. 아키텍처 개요 — 시스템 구성도, 데이터 흐름, 컴포넌트 관계
2. LLM 라우팅 — `quetta_ask/code/medical/multi_agent`
3. 스마트 디스패처 — `quetta_auto` 의도 분류
4. 원격 에이전트 — WebSocket 릴레이, GPU 자동 라우팅
5. 논문 분석 — Nougat + Gemini + Claude + RAG
6. 설계도 분석 — 기계/전기/CPLD PDF·PNG + RAG
7. 파일 업로드 & RAG — TUS + 파일 유형 감지 + 인제스트
8. 도구 레퍼런스 — 전체 도구 상세 파라미터
9. 환경변수 & 설정 — env 변수, 설치, 업데이트
10. 운영 가이드 — 로그, 트러블슈팅, 모니터링
11. 공유 메모리 & 멀티 계정 — RAG 지식베이스 공유, 세션 자동 초기화
12. 초대 토큰 설치 시스템 — 관리자 토큰 발급, 팀원 배포
13. 대화 히스토리 (NoSQL) — MongoDB 자동 저장, 계정별/통합 조회
14. 워크스페이스 (멀티 테넌트) — 업무/개발 분리, 사용자별 ACL, 접근 요청/승인

빠른 시작

```
# 1. 설치 (Mac/Linux 공통)
QUETTA_GATEWAY_URL=https://rag.quetta-soft.com \
QUETTA_API_KEY=your_api_key \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)

# 2. Claude Code 재시작

# 3. 사용
quetta_auto(request="이 논문 분석해줘", file_path="/data/paper.pdf")
quetta_auto(request="CPLD 설계도 해석", file_path="/data/cpld.pdf")
quetta_gpu_status()
```

전체 기능 맵

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                    Quetta Agents MCP                                                         │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                   │
    ┌──────────────────────────────┼──────────────────────────────────────────────────────────────────────┐
    │                              │                                                                      │
    ▼                              ▼                                                                      ▼
┌──────────────┐              ┌──────────────────────┐                                              ┌──────────────────────────┐
│ LLM          │              │ Remote               │                                              │ Document                 │
│ Gateway      │              │ Agent                │                                              │ Analysis                 │
└──────────────┘              └──────────────────────┘                                              └──────────────────────────┘
    │                              │                                                                      │
    ├── quetta_ask                 ├── install-link                                                       ├── paper (Nougat+Gemini+Claude)
    ├── quetta_code                ├── screenshot                                                         ├── blueprint (Gemini+Claude)
    ├── quetta_medical             ├── click/type/key                                                     ├── file_analysis (type-aware)
    ├── quetta_multi_agent         ├── shell (GPU auto)                                                   ├── auto RAG ingest
    └── quetta_routing_info        ├── gpu_exec/python                                                    └── upload_* (TUS)
                                   └── gpu_status

        ▲                                                                                                  │
        │                                                                                                  │
        └──────────────────────────── quetta_auto (의도 자동 분류) ──────────────────────────────────────┘
                                   │
                                   ▼
                          ┌──────────────────────────┐
                          │   RAG Store              │
                          │ (type=paper,             │
                          │  blueprint,              │
                          │  document,…)             │
                          └──────────────────────────┘
                                   │
                          ┌────────┴────────┐
                          ▼                 ▼
                quetta_paper_query   quetta_blueprint_query
```

버전

현재: **v0.11.0** (2026-04)

변경 이력은 ../README.md의 변경 이력 섹션 참고.

라이선스 & 링크

- Repository: https://github.com/choyunsung/quetta-agents-mcp
- Gateway: https://github.com/choyunsung/quetta-agents

01. 아키텍처 개요

Quetta Agents MCP는 **3계층 시스템**으로 구성됩니다.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│ Layer 1: Claude Code (사용자 인터페이스)                                    │
│   - stdio 기반 MCP 프로토콜로 연결                                          │
│   - quetta_*, quetta_auto 도구 호출                                        │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│ Layer 2: quetta-agents-mcp (클라이언트 라이브러리)                          │
│   - 도구 디스패처                                                            │
│   - 자연어 의도 분류 (quetta_auto)                                          │
│   - 파이프라인 오케스트레이션                                                │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│ Layer 3: Quetta 서버 (원격)                                                 │
│    ┌──────────────────────────┐   ┌──────────────────────────┐   ┌──────────────────────────────────────────────────────────
│    │ LLM Gateway  │   │ Orchestrator │   │ RAG Service      │   │
│    │ (스마트 라우팅)│   │ (에이전트/태스크)│  │ (벡터 검색·저장) │   │
│    └──────────────────────────┘   └──────────────────────────┘   └──────────────────────────────────────────────────────────
│                                                                    │
│    ┌──────────────────────────┐   ┌──────────────────────────────────────────────────────────────────────────────────────────
│    │ File Storage │   │ Vector DB + Metadata DB          │     │
│    │ (TUS upload) │   │ (RAG chunks + 인제스트 메타)       │     │
│    └──────────────────────────┘   └──────────────────────────────────────────────────────────────────────────────────────────
│                                                                    │
│   LLM 백엔드: 로컬 Gemma4/DeepSeek-R1 + Claude + Gemini            │
│                                                                    │
│    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│    │ Remote Agents (사용자의 외부 PC)                              │   │
│    │ - WebSocket 역방향 연결 (포트포워딩 불필요)                    │   │
│    │ - 화면 캡처 · 클릭 · 입력 · 셸                                 │   │
│    │ - GPU 작업 (Nougat OCR, torch, CUDA 등)                       │   │
│    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

컴포넌트별 역할

Layer 1: Claude Code

- MCP 클라이언트. 사용자가 자연어로 요청.
- 도구를 직접 호출하거나 `quetta_auto` 한 번으로 전체 위임.

Layer 2: quetta-agents-mcp

- 사용자의 PC에 설치되는 경량 클라이언트.
- **스마트 디스패처** `quetta_auto`: 자연어 요청을 의도별로 분류해 적절한 도구로 자동 라우팅.
- **파이프라인 오케스트레이션**: 논문/설계도 분석 같은 복합 작업을 여러 서비스에 걸쳐 순차 조율.
- 단계별 실패에 대해 graceful degradation (부분 성공 결과 반환).

Layer 3: Quetta 서버

서비스	역할
LLM Gateway	질문 유형에 따라 최적 모델 자동 선택 (Gemma4 / DeepSeek-R1 / Claude)
Orchestrator	전문 에이전트 관리, 태스크 실행, agent-skills 카탈로그
RAG Service	벡터 검색 + 인제스트 (논문/설계도/문서 통합 지식베이스)
File Storage	대용량 파일 재개 가능 업로드 (TUS 프로토콜)
Remote Agents	사용자의 외부 PC(GPU 포함)에 설치되는 에이전트

데이터 흐름 예시

예1: 논문 분석 + 이후 질의

```
사용자: "이 논문 분석해줘" + PDF 파일
  │
  ▼  MCP가 의도 자동 감지 → 논문 분석 파이프라인 실행
  │
  ├── 1. PDF 업로드
  ├── 2. GPU PC에서 Nougat OCR 실행 (수식·표 보존 Markdown)
  ├── 3. Gemini Vision 으로 그림·도표 해석
  ├── 4. Claude 가 전부 종합해 한글 리포트 생성
  └── 5. 결과를 지식베이스(RAG)에 자동 저장

이후:
사용자: "이 논문의 핵심 수식 유도 과정?"
  │
  ▼  MCP가 "논문 질의" 의도로 감지
  │
  └── 저장된 논문에서 관련 부분 검색 → Claude 답변 (출처 인용)
```

예2: 설계도 분석

```
사용자: "기계 조립도 GD&T 분석해줘" + PDF
  │
  ▼  MCP가 기계 설계 유형 자동 판별
  │
  ├── 1. PDF 텍스트 추출 (주석·치수·BOM)
  ├── 2. Gemini Vision 으로 GD&T 기호 · 조립 관계 해석
  ├── 3. Claude 엔지니어링 리포트 생성
  └── 4. 지식베이스 저장 → 이후 "M8 볼트 토크는?" 같은 질의 가능
```

성능 특성

작업	지연	비용
`quetta_ask` (단순 질문)	1–3초	Gemma4 (무료, 로컬)
`quetta_ask` (복잡)	5–15초	Claude Sonnet
`quetta_multi_agent`	30–90초	Gemma4 × 3 + Claude 종합
`quetta_remote_screenshot`	1–2초	무료
`quetta_gpu_exec`	커맨드 의존	무료 (로컬 GPU)
`quetta_analyze_paper` (nougat 설치됨)	3–10분	Gemini + Claude
`quetta_analyze_paper` (nougat 첫 설치)	+10분	—
`quetta_analyze_blueprint`	30–90초	Gemini + Claude
`quetta_paper_query` / `quetta_blueprint_query`	3–8초	Claude

장애 내성 (Fault Tolerance)

- **GPU 에이전트 단절 시**: 자동 재시도 후 명확한 에러 메시지 반환
- **분석 단계 중 일부 실패 시**: 나머지 단계로 계속 진행하여 부분 결과 반환
- **저장 실패 시**: 분석 결과는 그대로 반환 (사용자가 참고 가능)
- **원격 에이전트 연결 끊김 시**: 자동 재연결 (네트워크 복구 후)

보안

- 모든 외부 API 호출은 Bearer 토큰 / API Key 로 인증
- 원격 에이전트 설치 링크: **24시간** 유효, 일회성
- 원격 에이전트 WebSocket: 토큰 인증 필수
- 모든 데이터 전송: HTTPS (TLS)
- 사용자 계정별 API 키로 권한 분리 (관리자가 발급)

02. LLM 라우팅

Quetta Gateway(`:8701`)는 OpenAI 호환 `/v1/chat/completions` 엔드포인트를 제공하면서 **자동으로 최적 LLM**을 선택합니다.

라우팅 엔진

질문 특성	선택 모델	이유
단순 질문, 짧은 프롬프트	**Gemma4 (Ollama)**	로컬·무료·빠름
코드 작업	**Gemma4 + agent-skills**	5가지 skill 자동 주입
의료 임상/진단	**DeepSeek-R1**	의학 추론 특화
의료 영상 (CT/MRI/X-ray)	**Claude Opus**	비전 + 의학
복잡한 설계·아키텍처	**SCION 멀티에이전트**	Gemma4 × 3 병렬 → Claude 종합
4000+ 토큰, 논리적 복잡도 높음	**Claude Sonnet**	정확도

MCP 도구

`quetta_ask` — 만능 진입점

```
quetta_ask(query="React useEffect와 useLayoutEffect 차이?")
# → Gemma4 (일반 질문)

quetta_ask(query="복잡한 분산 시스템 설계...")
# → Claude Sonnet (복잡도 기준)
```

옵션: - `model` : `"auto"` (기본), `"gemma4"` , `"local"` , `"claude"` , `"claude-opus"` - `system_prompt` : 시스템 프롬프트 커스텀

`quetta_code` — 코드 작업 특화

agent-skills 5종 자동 주입 (plan/build/test/review/document)

```
quetta_code(
    task="이 함수를 TypeScript로 리팩토링",
    language="typescript",
    context="...원본 코드...",
)
```

`quetta_medical` — 의료 전문

```
quetta_medical(query="CRP 3.2, WBC 12000 패혈증 의심?")
quetta_medical(query="CT영상 분석", domain="imaging")  # Claude Opus
```

`quetta_multi_agent` — 복잡 태스크

```
quetta_multi_agent(task="ERP 마이크로서비스 아키텍처 설계")
# → Gemma4 × 3 에이전트 병렬 실행 → Claude 종합
```

`quetta_routing_info` — 미리 보기

```
quetta_routing_info(query="이 질문이 어디로 가는지 미리 알려줘")
# → 실제 호출 없이 라우팅 결정만 반환
```

응답 포맷

```
{
  "choices": [{"message": {"content": "..."}}],
  "routing": {
    "backend": "ollama",
    "model": "gemma4:27b",
    "reason": "short query, no code complexity",
    "injected_skills": ["plan", "build"],
    "multi_agent": false,
    "is_medical": false
  }
}
```

MCP는 이 routing 정보를 응답 하단에 메타로 표시:

```
---
_[Quetta] 모델: ollama (gemma4:27b)  |  이유: short query  |  스킬: plan, build_
```

모델 별칭

요청	실제 백엔드
`"auto"` (기본)	복잡도 기반 자동
`"gemma4"` / `"local"`	Ollama gemma4:27b
`"claude"` / `"claude-sonnet"`	claude-sonnet-4-6
`"claude-opus"`	claude-opus-4-6
`"deepseek-r1"`	DeepSeek-R1 (의료)

관련

- 스마트 디스패처 — 자연어 → 도구 자동 선택
- 도구 레퍼런스

03. 스마트 디스패처 (`quetta_auto`)

사용자의 자연어 요청을 **의도 분류**해 자동으로 적절한 도구로 라우팅합니다.

개념

```
User: "이 CPLD 도면 분석" + file_path
        │
        ▼
[quetta_auto] ── 키워드 매칭 → intent=blueprint_analysis
        │
        ▼
    call_tool("quetta_analyze_blueprint", args)
        │
        ▼
    (pipeline execution)
```

의도 분류 (우선순위 순)

의도	키워드 예시	라우팅
blueprint_query	"저장된 설계도", "인제스트된 도면"	`quetta_blueprint_query`
blueprint_analysis	"설계도", "도면", "blueprint", "cpld", "회로도"	`quetta_analyze_blueprint`
paper_query	"저장된 논문", "분석된 논문에서"	`quetta_paper_query`
paper_analysis	"논문 분석", ".pdf", "arxiv", "학술"	`quetta_analyze_paper`
gpu_compute	"cuda", "torch", "nvidia-smi", "train.py"	`quetta_gpu_exec`
screenshot	"스크린샷", "화면 캡처"	`quetta_remote_screenshot`
remote_shell	"원격 명령", "원격 셸", "원격 실행"	`quetta_remote_shell`
file_analysis	"파일 분석", "의료 데이터", ".csv"	`quetta_analyze_file`
medical	"진단", "증상", "ICD", "환자"	`quetta_medical`
code	"리팩토링", "버그 수정", "코드 작성"	`quetta_code`
multi_agent	"설계", "아키텍처", "복잡한"	`quetta_multi_agent`
question (기본값)	그 외	`quetta_ask`

파라미터

```
quetta_auto(
    request="자연어 요청",   # 필수
    agent_id="",            # 선택 - GPU/remote 의도 시 특정 에이전트
    file_path="",           # 선택 - file_analysis/paper/blueprint 의도 시
    dry_run=False,          # true면 실행 없이 분류 결과만
)
```

사용 예시

```
# 논문
quetta_auto(request="이 논문 분석해줘", file_path="/data/paper.pdf")

# 설계도 (타입 자동 추정)
quetta_auto(request="전기 결선도 해석", file_path="/data/power.pdf")
# → blueprint_analysis, drawing_type=electrical

quetta_auto(request="CPLD 스키매틱 분석", file_path="/data/cpld.pdf")
# → blueprint_analysis, drawing_type=cpld

# GPU 작업
quetta_auto(request="nvidia-smi 상태")
quetta_auto(request="python train.py --epochs 100")

# 원격 제어
quetta_auto(request="화면 보여줘")

# 의료
quetta_auto(request="CRP 3.2 의미")

# 저장된 자료 질의
quetta_auto(request="저장된 논문에서 attention 비교")
quetta_auto(request="인제스트된 설계도 목록")

# 분류 확인만
quetta_auto(request="이게 뭐로 갈까?", dry_run=True)
# → **[quetta_auto]** 의도: `question`
```

한국어/영어 혼용

키워드는 한글·영문을 모두 포함합니다: - `"논문 분석"` / `"paper analysis"` → paper_analysis - `"설계도"` / `"blueprint"` / `"schematic"` → blueprint_analysis - `"코드 리뷰"` / `"refactor"` → code

코드 블록 추출

백틱으로 감싼 명령어/코드는 자동 추출됩니다:

```
quetta_auto(request="이거 GPU에서 실행: `python -c 'import torch; print(torch.cuda.is_available())'`")
# → gpu_compute → quetta_gpu_exec(command="python -c 'import torch; print(torch.cuda.is_available())'")
```

응답 형식

```
**[quetta_auto]** 의도: `blueprint_analysis`  |  매칭: `설계도, cpld`

(하위 도구 실행 결과)
```

dry_run:

```
**[quetta_auto]** 의도: `paper_analysis`  |  매칭: `논문 분석, .pdf`

_(dry_run - 실행 생략)_
```

04. 원격 에이전트 (Remote Agent)

다른 PC(GPU 서버, 개인 컴퓨터)를 원격 제어합니다. 에이전트가 서버로 **역방향 WebSocket**을 연결하므로 포트포워딩/방화벽 설정 불필요.

아키텍처

```
[Claude MCP] ──▶ [Quetta Gateway] ──▶ [Remote Agent PC]
                                      (역방향 WebSocket 연결)
```

- 에이전트가 서버로 **역방향 연결**하므로 사용자 PC에 별도 포트 개방 불필요
- 네트워크 단절 시 자동 재연결
- 24시간 유효 설치 링크

에이전트 설치

방법 1: `/remote-agent` 스킬 (권장)

Claude Code에서:

```
/remote-agent
```

→ 설치 링크 표시 + 연결 자동 감지 (최대 10분 대기)

방법 2: 수동

```
# Linux/Mac
curl -fsSL "https://rag.quetta-soft.com/agent/download?token=XXX&os=linux" | bash

# Windows (PowerShell 또는 CMD)
# 브라우저에서 install-quetta-agent.bat 다운로드 후 실행
```

설치 과정

1. Python 3.9+ 확인
2. `websockets`, `pyautogui`, `mss`, `pillow`, `pyperclip` 설치
3. `~/.quetta-remote-agent/agent.py` 생성 + `.env`에 URL/토큰 저장
4. 에이전트 즉시 실행 (WS 연결)

연결된 에이전트 확인

```
quetta_remote_connect(action="list")
# → ID / 호스트 / OS / GPU / 화면제어 / 스크린샷
```

원격 제어 도구

도구	파라미터	설명
`quetta_remote_screenshot`	`agent_id`, `max_width?`	화면 캡처 (Claude가 이미지 분석)
`quetta_remote_click`	`x, y, button?, double?`	마우스 클릭
`quetta_remote_type`	`text, interval?`	텍스트 입력 (클립보드 경유)
`quetta_remote_key`	`key` (e.g. `ctrl+c`)	단축키
`quetta_remote_shell`	`command, timeout?, cwd?`	셸 명령 (GPU 키워드 자동)

GPU 자동 라우팅

명령어에 **GPU 키워드**(cuda, torch, nvidia-smi, train.py 등)가 포함되면 agent_id 미지정 시 자동으로 GPU 에이전트 선택:

```
# agent_id 없이도 GPU 에이전트 자동 선택
quetta_remote_shell(command="nvidia-smi")
quetta_gpu_exec(command="python train.py --epochs 100")
quetta_gpu_python(code="""
import torch
print(torch.cuda.get_device_name(0))
""")
```

`_pick_agent()` **선택 로직:** 1. `agent_id` 명시 → 사용 2. GPU 키워드 감지 or `prefer_gpu=True` → GPU 에이전트 자동 3. `REMOTE_AGENT_ID` 환경변수 → 사용 4. 연결 에이전트가 1개뿐 → 자동 5. GPU 필요한데 없음 → 설치 링크 반환 후 에러

Windows 백그라운드 서비스

설치 후 로그온 시 자동 시작되도록 구성할 수 있습니다 (콘솔 창 없이 실행). 설치 스크립트가 자동 생성한 VBS 파일을 시작프로그램 폴더에 복사하면 됩니다.

보안

- 설치 링크: **24시간** 유효, 1회성
- 원격 에이전트 연결: 토큰 인증
- 모든 통신: TLS 암호화

트러블슈팅

자주 발생하는 문제는 10. 트러블슈팅 참고.

관련 문서

- 스마트 디스패처
- 도구 레퍼런스

05. 논문 분석 (Nougat + Gemini + Claude + RAG)

학술 논문 PDF를 **3단 AI 파이프라인**으로 완벽 분석하고, 결과를 RAG 지식베이스에 자동 저장해 이후 질의할 수 있게 합니다.

파이프라인

```
PDF 입력
    │
    ├── 1. 파일 업로드 (대용량 지원, 재개 가능)
    │
    ├── 2. Nougat OCR (GPU 가속)
    │     PDF → 수식·표·구조가 보존된 Markdown
    │
    ├── 3. Gemini Vision
    │     Figure / Table / 수식 시각 해석
    │
    ├── 4. Claude 종합
    │     Nougat + Gemini 통합 → 한글 엔지니어링 리포트
    │     (제목·저자·초록·방법·결과·한계)
    │
    └── 5. 지식베이스 자동 저장
        섹션·청크 단위로 나누어 저장 → 이후 질의 시 자동 참조
```

도구

`quetta_analyze_paper`

입력: - `file_path` (string): 서버 로컬 PDF 경로 (또는 `file_id`) - `file_id` (string): 이미 TUS 업로드된 PDF ID - `query` (string, optional): 집중 분석할 초점 - `agent_id` (string, optional): 사용할 GPU 에이전트 (미지정 시 자동) - `install_nougat` (bool, default=True): nougat 미설치 시 자동 설치 - `skip_gemini` (bool, default=False): Gemini 단계 건너뜀 - `skip_claude` (bool, default=False): Claude 종합 건너뜀 - `ingest_to_rag` (bool, default=True): RAG 자동 저장 - `tags` (array): RAG 메타 태그

출력: - 진행 로그 (1/4 ~ 4/4) - Claude 종합 리포트 (한글) - RAG 인제스트 요약 (청크 수) - 접을 수 있는 섹션: - Nougat 원본 추출 (처음 8000자) - Gemini 시각 분석 (처음 6000자)

`quetta_paper_query`

입력: - `query` (string): 논문 관련 질문 - `filename` (string, optional): 특정 논문만 필터 - `list` (bool, default=False): 질의 대신 목록만 - `top_k` (int, default=8): 참조할 청크 수

출력: - RAG 검색 결과 기반 Claude 답변 (출처 인용) - 목록 모드 시: 인제스트된 논문 이름/청크 수/file_id

사용 예시

기본 흐름

```
# 1. 로컬 PDF 분석 + RAG 자동 저장
quetta_analyze_paper(file_path="/data/papers/attention.pdf")

# 결과 (요약):
# ## 📄 논문 분석 파이프라인
# **1/4**  파일 로드: attention.pdf (2,134,567 bytes)
# **2/4**  TUS 업로드 완료: a1b2c3d4
# **3/4**  GPU 에이전트 선택: 878047e4
#   → nougat 추출 완료: 45,231 chars
# **4/4**  Gemini CLI 분석 완료: 12,450 chars
#
# ---
#
# ## 🎯 종합 분석 (Claude)
# # Attention Is All You Need (Vaswani et al., 2017)
# ## 제목 / 저자 ...
# ## 초록 한글 요약 ...
# ## 핵심 기여 ...
# ...
#
# **RAG 인제스트 완료**
# - Source 태그: `paper:attention.pdf`
# - Nougat 본문 청크: 42개
# - Claude 종합: 1
# - Gemini 시각분석: 1
```

저장된 논문 질의

```
# 인제스트된 논문 목록
quetta_paper_query(list=True)

# 특정 논문 내 질의 (filename 필터)
quetta_paper_query(
    query="scaled dot-product attention의 수식 유도",
    filename="attention.pdf",
)

# 전체 논문 대상 비교 질의
quetta_paper_query(query="어떤 논문들이 self-attention을 제안했나?")
```

자연어 디스패처 활용

```
# 자동 라우팅
quetta_auto(request="이 논문 분석해줘", file_path="/data/paper.pdf")
# → paper_analysis 의도 감지 → quetta_analyze_paper

quetta_auto(request="저장된 논문에서 Transformer 비교해줘")
# → paper_query 의도 감지 → quetta_paper_query
```

옵션별 사용 시나리오

```
# Gemini 건너뛰고 빠르게 (Nougat + Claude만)
quetta_analyze_paper(
    file_path="/data/paper.pdf",
    skip_gemini=True,
)

# Nougat 설치 생략 (이미 설치된 경우)
quetta_analyze_paper(
    file_path="/data/paper.pdf",
    install_nougat=False,
)

# 분석만 하고 RAG에 저장하지 않음 (1회성)
quetta_analyze_paper(
    file_path="/data/paper.pdf",
    ingest_to_rag=False,
)

# 특정 GPU 에이전트 지정
quetta_analyze_paper(
    file_path="/data/paper.pdf",
    agent_id="878047e4",
)

# 태그와 함께 저장
quetta_analyze_paper(
    file_path="/data/paper.pdf",
    tags=["transformer", "NLP", "2017", "required-reading"],
)
```

필요 환경

구성 요소	설치	비고
GPU 에이전트	`/remote-agent` 스킬로 설치	Nougat는 torch + CUDA 필요
Nougat (choyunsung/nougat)	자동 설치 (`install_nougat=True`)	첫 실행 시 최대 10분 소요
gemini CLI	`npm i -g @google/gemini-cli` + `gemini` 로 OAuth 로그인	선택 (미설치 시 자동 건너뜀)
Gateway `QUETTA_API_KEY`	`.env` 에 설정	외부 접근 시 필수
RAG 서비스	Quetta 서버에서 자동 실행	`:8400`

저장 구조

인제스트된 논문은 아래와 같이 구분되어 저장됩니다:

- **본문**: 섹션·청크 단위로 여러 개 저장
- **종합 리포트**: 1개 (Claude)

- **시각 분석**: 1개 (Gemini)

각각 `type=paper`, `filename`, `tags` 로 식별 가능해 이후 특정 논문/태그로 필터링된 검색이 가능합니다.

성능·비용

단계	소요 시간	비고
TUS 업로드	1–5초	PDF 크기 의존
Nougat (첫 실행)	+5–10분	pip install + 모델 다운로드
Nougat (2회차+)	1–5분	PDF 페이지 수 의존 (~10초/page)
Gemini 분석	30–90초	PDF 크기 의존
Claude 종합	10–30초	본문 길이 의존
RAG 인제스트	5–15초	청크 수 의존

전체 평균: 3–10분 (첫 실행은 10–20분)

트러블슈팅

"GPU 에이전트가 없음"

Claude Code 채팅에서 `/remote-agent` 실행 → GPU PC에 에이전트 설치 후 재시도.

Gemini 건너뛰기 원할 때

```
quetta_analyze_paper(file_path="...", skip_gemini=True)
```

자주 발생하는 문제

10. 트러블슈팅 참고.

관련 문서

- 원격 에이전트
- 설계도 분석
- 파일 업로드 & RAG

06. 설계도 분석 (기계 / 전기 / CPLD)

기계·전기·CPLD 설계도(PDF·PNG)를 **Gemini Vision + PyMuPDF + Claude**로 분석하고 RAG에 자동 저장합니다.

파이프라인

```
PDF/PNG 입력
    |
    ├──[1] 파일 로드 + 확장자 판별 (pdf/png/jpg)
    |      + TUS 업로드 (RAG 인제스트 연결용)
    |
    ├──[2] PDF면 PyMuPDF로 벡터 텍스트 추출
    |      - 주석, 치수, BOM, 기호, 핀맵 등
    |      - 이미지 파일이면 이 단계 생략
    |
    ├──[3] Gemini CLI Vision 분석 (타입별 전문 프롬프트)
    |      - mechanical / electrical / cpld / auto
    |      - @filepath 구문으로 PDF/PNG 직접 전달
    |
    ├──[4] Claude Sonnet 종합
    |      - 벡터 텍스트 + 시각 분석 통합
    |      - 엔지니어링 실무 리포트 생성
    |
    └──[5] RAG 자동 인제스트
         - type=blueprint, drawing_type=<type>
         - part=pdf_text / vision / synthesis
         - 이후 quetta_blueprint_query로 재질의
```

타입별 전문 프롬프트

기계 설계 (`mechanical`)

분석 항목: 1. 도면 종류 (조립도/부품도/단면도/상세도) 2. 주요 치수·공차·표면거칠기(Ra)·끼워맞춤 기호 3. 재질·열처리·표면처리 기재사항 4. 각 부품의 기능과 조립 관계 5. **GD&T 기호** 해석 — ⊥, ⌒, ◎, ⊕, ↕ 등 6. 제작/가공 시 주의사항 (가공 순서·기준면) 7. BOM(부품표) 항목별 정리

전기 설계 (`electrical`)

분석 항목: 1. 회로 종류 (배전/제어/PLC I/O/시퀀스/단선결선도) 2. 주요 소자 (차단기, 계전기, 인버터, PLC, 센서) 목록·정격 3. 전원 계통 (전압·상·주파수)과 결선 4. 신호 흐름 — 입력/출력/인터록·안전회로 5. 부하/모터 용량·보호장치 설정 6. 단자번호·와이어 번호 규칙 7. 안전 관련 (ESTOP, 접지, 절연) 확인

CPLD / FPGA / 디지털 논리 (`cpld`)

분석 항목: 1. 설계 유형 (RTL 블록도/스키매틱/타이밍도/상태천이도/핀맵) 2. 주요 모듈·서브 블록과 기능 3. 신호 이름·비트폭·방향(input/output/inout) 4. 클럭·리셋 계통 (동기/비동기, 클럭 도메인) 5. **FSM** 상태와 전이 조건 6. 인터페이스 프로토콜 (I2C/SPI/UART/AXI 등) 7. 타이밍 제약 (setup/hold, tPD) 및 합성 가능 여부 8. 가능하면 **Verilog/VHDL 구조 유추**

자동 감지 (`auto`)

입력 도면의 종류를 먼저 판별한 뒤 해당 타입에 맞는 분석 수행.

도구

`quetta_analyze_blueprint`

입력: - `file_path` (string): 서버 로컬 PDF/PNG 경로 - `file_id` (string): 이미 TUS 업로드된 파일 ID - `drawing_type` (enum): `mechanical` / `electrical` / `cpld` / `auto` - `query` (string, optional): 집중 분석 포인트 - `tags` (array): RAG 메타 태그 - `ingest_to_rag` (bool, default=True): RAG 자동 저장 - `skip_gemini` (bool, default=False): Gemini 단계 생략 (Claude만)

출력: - 진행 로그 (1/3 ~ 3/3) - Claude 엔지니어링 리포트 (한글, 타입별 구조화) - RAG 인제스트 요약 - 접을 수 있는 섹션: - Gemini 시각 분석 원본 (8000자) - PDF 벡터 텍스트 (6000자)

`quetta_blueprint_query`

입력: - `query` (string): 질문 - `filename` (string, optional): 특정 도면만 - `drawing_type` (string, optional): 특정 타입만 (`mechanical` 등) - `list` (bool, default=False): 목록 모드 - `top_k` (int, default=8): 참조 청크 수

출력: - RAG 검색 기반 Claude 답변 (치수/부품번호 원문 인용)

사용 예시

기본 분석

```
# 기계 설계도 분석
quetta_analyze_blueprint(
    file_path="/data/drawings/gear_assembly.pdf",
    drawing_type="mechanical",
    tags=["gearbox", "v2.0"],
)

# 전기 단선결선도
quetta_analyze_blueprint(
    file_path="/data/drawings/power_dist.pdf",
    drawing_type="electrical",
)

# CPLD 스키매틱
quetta_analyze_blueprint(
    file_path="/data/drawings/cpld_top.pdf",
    drawing_type="cpld",
    query="FSM 상태와 전이 조건을 Verilog로 유추해줘",
)

# 자동 감지
quetta_analyze_blueprint(file_path="/data/drawings/unknown.png")

# PNG 이미지 도면
quetta_analyze_blueprint(
    file_path="/data/drawings/schematic.png",
    drawing_type="electrical",
)
```

자연어 디스패처

```
# 자동 라우팅 + 타입 추정
quetta_auto(request="이 CPLD 설계도 해석", file_path="/data/cpld.pdf")
# → blueprint_analysis, drawing_type=cpld 자동 선택

quetta_auto(request="기계 조립도 GD&T 분석", file_path="/data/asm.pdf")
# → blueprint_analysis, drawing_type=mechanical

quetta_auto(request="저장된 설계도에서 모터 회로 찾아줘")
# → blueprint_query
```

저장된 설계도 질의

```
# 목록
quetta_blueprint_query(list=True)
# → ## 인제스트된 설계도 (3개)
#    - gear_assembly.pdf [mechanical] 청크 5개, file_id: abc123
#    - power_dist.pdf    [electrical] 청크 3개, file_id: def456
#    - cpld_top.pdf      [cpld]       청크 4개, file_id: ghi789

# 특정 도면
quetta_blueprint_query(
    query="M8 볼트 개수와 체결 토크",
    filename="gear_assembly.pdf",
)

# 특정 타입만
quetta_blueprint_query(
    query="FSM 리셋 조건",
    drawing_type="cpld",
)

# 전체 대상
quetta_blueprint_query(query="ESTOP 회로가 있는 도면?")
```

출력 예시 (실제)

```
## 📐 설계도 분석 파이프라인 (drawing_type: cpld)

**1/3**  파일: cpld_top.pdf (456,789 bytes)
  → TUS 업로드: abc123xyz
**2/3**  PDF 텍스트 추출: 3,421 chars
**3/3**  Gemini 시각 분석: 8,945 chars

---

## 🔧 엔지니어링 분석 리포트 (cpld)

### 1. 도면 개요
- 제목: CPLD TOP LEVEL SCHEMATIC
- 도면번호: DSGN-2026-04-12
- 축척: N/A (논리 스키매틱)

### 2. 종류와 용도
RTL 블록도 - I/O 제어용 CPLD (Xilinx XC9572XL-10VQG64C)
... (중략)

### 3. 핵심 사양
| 항목 | 값 |
|------|-----|
| 클럭 | 25 MHz (CLK_MAIN), 1 MHz (CLK_SPI) |
| I/O | 48 (34 available) |
| 전압 | 3.3V I/O, 3.3V core |

### 4. 부품/블록 리스트
| 이름 | 역할 | 포트 |
|------|------|------|
| U1   | CPLD (XC9572XL) | 64pin VQFP |
| Y1   | 25MHz OSC      | → CLK_MAIN |
...

### 5. 신호 흐름
- CLK_MAIN → PLL_DIV → {CLK_25, CLK_5}
- RST_N (비동기) → sync_reset_module → RST_SYNC
- SPI_CS ↔ spi_slave module (state=IDLE→CMD→ADDR→DATA)
...

**RAG 인제스트 완료** (source: `blueprint:cpld_top.pdf`)
- 벡터 텍스트 청크: 2개
- Gemini 시각 분석: 1
- Claude 종합: 1
```

저장 구조

- **PDF 텍스트**: 청크 단위 (여러 개)
- **시각 분석**: 1개 (Gemini)
- **종합 리포트**: 1개 (Claude)

각각 `type=blueprint`, `drawing_type` (mechanical/electrical/cpld), `filename`, `tags` 로 식별되어 이후 필터 검색이 가능합니다.

필요 환경

구성 요소	설치	비고
PyMuPDF	자동 설치 (pip `pymupdf>=1.24`)	MCP dependency
gemini CLI	`npm i -g @google/gemini-cli` + OAuth	미설치 시 Claude만
Gateway `QUETTA_API_KEY`	`.env` 설정	외부 접근 시 필수

GPU 에이전트 불필요 — 논문 분석과 달리 Nougat를 사용하지 않습니다.

지원 파일 형식

확장자	지원	비고
`.pdf`	✅	벡터 텍스트 + Vision
`.png`	✅	Vision만
`.jpg` / `.jpeg`	✅	Vision만
`.dwg` / `.dxf`	❌	별도 변환 필요 (ezdxf) — 추후 추가 예정
`.svg`	❌	추후 추가 예정

성능

작업	시간
PDF 텍스트 추출	<1초
Gemini 시각 분석	20–60초
Claude 종합	10–30초
RAG 인제스트	2–5초
전체	**30–90초**

타입 추정 로직 (`quetta_auto`)

자연어 요청에서 `drawing_type` 자동 판단: - `cpld`, `fpga`, `pcb`, `verilog`, `vhdl`, `rtl` → `cpld` - `전기`, `회로도`, `결선`, `plc`, `배전`, `control` → `electrical` - `기계`, `gd&t`, `조립`, `부품도`, `공차`, `치수` → `mechanical` - 매칭 없음 → `auto`

트러블슈팅

"지원하지 않는 형식"

PDF / PNG / JPG 만 지원됩니다. DWG/DXF 는 사전에 PDF/PNG 로 변환 필요.

"PDF 텍스트 없음 (이미지 도면으로 처리)"

스캔본 PDF인 경우 정상 동작입니다 — Gemini Vision 만으로도 상세 분석이 이루어집니다.

자세한 내용은 10. 트러블슈팅 참고.

확장 계획

- [ ] `.dwg` / `.dxf` 지원 (ezdxf로 벡터 추출)
- [ ] 대형 A0 도면 자동 타일링 (OpenCV)
- [ ] 회로도 → Verilog 자동 코드 생성 (CPLD)
- [ ] P&ID (공정배관계장도) 특화 프롬프트
- [ ] BOM CSV 자동 추출

관련 문서

- 논문 분석
- 파일 업로드 & RAG

07. 파일 업로드 & RAG 통합

대용량 파일 업로드 (TUS) + 유형 자동 감지 + RAG 지식베이스 인제스트.

데이터 흐름

```
로컬 파일
    |
    ├──[1] quetta_upload_file
    |      ├── TUS 프로토콜로 tusd에 청크 업로드
    |      └── file_id 반환
    |
    ├──[2] quetta_analyze_file (자동 유형 감지)
    |      ├── RAG /upload/analyze/{file_id}
    |      ├── 유형 분기:
    |      |     - medical → DeepSeek-R1 분석
    |      |     - signal_data → Gemma4 분석
    |      |     - document → Gemma4 요약
    |      └── 자동 RAG 인제스트
    |
    ├──[3] quetta_upload_process (수동 인제스트)
    |      ├── RAG /upload/process/{file_id}
    |      └── usage_type 명시 지정
    |
    └──[4] quetta_upload_process_all
         └── 미처리 파일 일괄 인제스트
```

유형 자동 감지

RAG 서비스가 파일명·내용 기반으로 판별:

유형	감지 기준	처리
`medical`	의료 키워드 ≥2개 (환자/진단/ICD/FHIR 등)	medical 네임스페이스 + DeepSeek-R1
`signal_data`	EDF/DAT/MAT/HDF5 확장자, ECG/BPM 헤더	signal_data 네임스페이스
`document`	PDF/DOCX/TXT 등	documents 네임스페이스

도구

`quetta_upload_file`

```
# 로컬 파일
quetta_upload_file(file_path="/data/report.pdf")
# → file_id: abc123, size: 2MB

# 텍스트 직접
quetta_upload_file(
    content="환자 65세 남성, CRP 3.2...",
    filename="case.txt",
)
```

`quetta_upload_list`

```
quetta_upload_list()
# → **업로드된 파일 목록** (12개)
#    - `abc123` – report.pdf
#      크기: 2,345,678 bytes | 상태: 완료
#    ...
```

`quetta_upload_process`

```
quetta_upload_process(
    file_id="abc123",
    usage_type="measurement_data",  # 또는 medical/document
    source="실험1 결과",
    tags=["v2.0"],
    chunk_size=4000,
)
```

`quetta_upload_process_all`

```
quetta_upload_process_all()
# 미처리 파일 전체 자동 인제스트
```

`quetta_analyze_file` (올인원)

업로드 + 유형 감지 + RAG + AI 분석을 한 번에:

```
quetta_analyze_file(file_path="/data/patient.csv")

# 결과:
# **파일 유형:** medical (환자, ICD 등 5개 키워드 매칭)
# **저장 경로:** /storage/uploads/tusd/abc123
# **RAG 인제스트:** 8개 청크
#
# ## 🩺 의료 데이터 분석 (DeepSeek-R1)
# ...
```

저장 구조

모든 인제스트는 공통 분류 체계를 가집니다:

- `type` : `paper` / `blueprint` / `medical` / `document` / `signal_data`
- `filename` : 원본 파일명
- `tags` : 사용자 지정 태그 (검색용)
- `part` : 내용 종류 (본문 / 종합 / 시각 분석)

이 메타데이터 덕분에 이후 `quetta_paper_query` , `quetta_blueprint_query` 등으로 **특정 파일·타입·태그** 필터 검색이 가능합니다.

질의 계층

도구	용도
`quetta_paper_query`	`type=paper` 필터
`quetta_blueprint_query`	`type=blueprint` 필터
`quetta_ask`	RAG harness (gateway 내부 통합) — 전체 검색

Gateway RAG Harness: `quetta_ask` 는 gateway의 RAG harness를 통해 답변에 RAG 컨텍스트를 자동 주입합니다 (필요 시). 즉 논문·도면 인제스트만으로도 `quetta_ask("X 논문의 Y는?")` 로 답변 가능.

대용량 파일 업로드

업로드는 **재개 가능한 프로토콜**로 처리됩니다: - 네트워크 중단 시 중단 지점부터 이어서 업로드 - 수 GB 파일도 안정적으로 전송 - 클라이언트에서 내부적으로 자동 처리 — 사용자는 `quetta_upload_file` 만 호출하면 됨

트러블슈팅

자주 발생하는 문제는 10. 트러블슈팅 참고.

관련 문서

- 논문 분석
- 설계도 분석
- 도구 레퍼런스

08. 도구 레퍼런스

Quetta Agents MCP가 제공하는 모든 도구의 상세 파라미터.

🎯 스마트 디스패처

`quetta_auto`

자연어 요청을 자동 분류해 적절한 도구로 라우팅.

파라미터	타입	기본	설명
`request`	string	—	자연어 요청 (필수)
`agent_id`	string	`""`	(GPU/remote 의도 시)
`file_path`	string	`""`	(file/paper/blueprint 의도 시)
`dry_run`	bool	`false`	실행 없이 분류만

🧠 LLM 게이트웨이

`quetta_ask`

파라미터	타입	기본	설명
`query`	string	—	질문 (필수)
`model`	string	`"auto"`	`auto` / `gemma4` / `claude` / `claude-opus`
`system_prompt`	string	`""`	커스텀 시스템 프롬프트

`quetta_code`

파라미터	타입	기본	설명
`task`	string	—	코딩 작업 설명 (필수)
`language`	string	`""`	프로그래밍 언어
`context`	string	`""`	관련 코드

`quetta_medical`

파라미터	타입	기본	설명
`query`	string	—	의료 질문 (필수)
`domain`	string	`"auto"`	`auto` / `imaging` / `clinical` / `pharmacy`

`quetta_multi_agent`

파라미터	타입	기본	설명
`task`	string	—	복잡 태스크 (필수)

`quetta_routing_info`

파라미터	타입	기본	설명
`query`	string	—	라우팅 예측할 질문

`quetta_list_agents`

파라미터 없음 — 등록된 에이전트 목록.

`quetta_run_agent`

파라미터	타입	기본	설명
`agent_name`	string	—	에이전트 이름
`task`	string	—	위임할 태스크

💻 원격 제어

`quetta_remote_connect`

파라미터	타입	기본	설명
`action`	enum	`"list"`	`list` / `install-link`
`os`	enum	`"linux"`	`linux` / `windows` / `mac`

`quetta_remote_screenshot`

파라미터	타입	기본	설명
`agent_id`	string	`""`	(미지정 시 자동)
`max_width`	int	`1280`	리사이즈 최대폭

`quetta_remote_click`

파라미터	타입	기본	설명
`agent_id`	string	`""`	
`x`	int	—	필수
`y`	int	—	필수
`button`	enum	`"left"`	`left` / `right` / `middle`
`double`	bool	`false`	더블클릭

`quetta_remote_type`

파라미터	타입	기본	설명
`agent_id`	string	`""`	
`text`	string	—	입력할 텍스트 (필수)
`interval`	float	`0.03`	글자 간 지연(sec)

`quetta_remote_key`

파라미터	타입	기본	설명
`agent_id`	string	`""`	
`key`	string	—	e.g. `ctrl+c` , `alt+tab`

`quetta_remote_shell`

파라미터	타입	기본	설명
`agent_id`	string	`""`	
`command`	string	—	필수
`timeout`	int	`30`	초
`cwd`	string	`""`	작업 디렉토리
`prefer_gpu`	bool	`false`	GPU 에이전트 강제

🚀 GPU 자동 라우팅

`quetta_gpu_exec`

파라미터	타입	기본	설명
`command`	string	—	필수
`agent_id`	string	`""`	(미지정 시 GPU 자동)
`timeout`	int	`300`	
`cwd`	string	`""`	

`quetta_gpu_python`

파라미터	타입	기본	설명
`code`	string	—	Python 코드 (필수)
`agent_id`	string	`""`	
`timeout`	int	`300`	
`python`	string	`"python"`	실행 파일

`quetta_gpu_status`

파라미터 없음 — 모든 GPU 에이전트 `nvidia-smi` 요약.

📑 논문 분석

`quetta_analyze_paper`

파라미터	타입	기본	설명
`file_path`	string	`""`	로컬 PDF
`file_id`	string	`""`	TUS file_id
`query`	string	`""`	초점
`agent_id`	string	`""`	GPU 에이전트
`install_nougat`	bool	`true`	자동 설치
`skip_gemini`	bool	`false`	
`skip_claude`	bool	`false`	
`ingest_to_rag`	bool	`true`	RAG 저장
`tags`	array	`[]`	

`quetta_paper_query`

파라미터	타입	기본	설명
`query`	string	`""`	
`filename`	string	`""`	특정 논문만
`list`	bool	`false`	목록 모드
`top_k`	int	`8`	

📐 설계도 분석

`quetta_analyze_blueprint`

파라미터	타입	기본	설명
`file_path`	string	`""`	PDF/PNG/JPG
`file_id`	string	`""`	
`drawing_type`	enum	`"auto"`	`mechanical` / `electrical` / `cpld` / `auto`
`query`	string	`""`	
`tags`	array	`[]`	
`ingest_to_rag`	bool	`true`	
`skip_gemini`	bool	`false`	

`quetta_blueprint_query`

파라미터	타입	기본	설명
`query`	string	`""`	
`filename`	string	`""`	
`drawing_type`	string	`""`	필터
`list`	bool	`false`	
`top_k`	int	`8`	

📁 파일 업로드 & 분석

`quetta_analyze_file`

파라미터	타입	기본	설명
`file_path`	string	`""`	
`content`	string	`""`	텍스트 직접
`filename`	string	`"upload.txt"`	
`query`	string	`""`	
`source`	string	`""`	RAG 소스 태그
`tags`	array	`[]`	

`quetta_upload_file`

파라미터	타입	기본	설명
`file_path`	string	`""`	
`content`	string	`""`	
`filename`	string	`"upload.txt"`	

`quetta_upload_list`

파라미터 없음.

`quetta_upload_process`

파라미터	타입	기본	설명
`file_id`	string	—	필수
`usage_type`	string	`"measurement_data"`	
`source`	string	`""`	
`tags`	array	`[]`	
`chunk_size`	int	`4000`	

`quetta_upload_process_all`

파라미터 없음.

🔧 버전 관리

`quetta_version`

파라미터 없음 — 현재 버전 + GitHub 최신 커밋.

`quetta_update`

파라미터 없음 — GitHub 최신으로 자동 업데이트.

반환 타입

대부분 도구는 `TextContent` 배열 반환. 예외: - `quetta_remote_screenshot` : `[ImageContent, TextContent]` — Claude가 이미지 직접 분석 - `quetta_analyze_paper` / `quetta_analyze_blueprint` : 마크다운 리포트 (접을 수 있는 `<details>` 포함)

에러 핸들링

- GPU 에이전트 없음: 설치 링크 포함한 에러
- TUS 업로드 실패: 네트워크 에러 메시지
- RAG 인제스트 실패: 경고만 표시, 분석 결과는 유지
- WebSocket 타임아웃: 자동 재시도 (3회)

09. 설치 & 설정

설치

원클릭 설치 (Mac / Linux)

```
curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh | bash
```

환경변수로 미리 지정 가능:

```
QUETTA_GATEWAY_URL=https://rag.quetta-soft.com \
QUETTA_API_KEY=발급받은_API_키 \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)
```

install.sh가 자동으로: 1. `uv` 미설치 시 설치 2. GitHub에서 패키지 다운로드 및 검증 3. `claude mcp add-json` CLI로 등록 (user scope) 4. Gateway URL이 `https://` 면 RAG/TUSD URL 자동 설정

수동 설치

```
claude mcp add-json quetta-agents '{
  "command": "uvx",
  "args": ["--from", "git+https://github.com/choyunsung/quetta-agents-mcp", "quetta-agents-mcp"],
  "env": {
    "QUETTA_GATEWAY_URL": "https://rag.quetta-soft.com",
    "QUETTA_ORCHESTRATOR_URL": "https://rag.quetta-soft.com/orchestrator",
    "QUETTA_API_KEY": "발급받은_API_키"
  }
}' --scope user
```

설치 확인

```
claude mcp list | grep quetta
```

환경변수

변수	기본값	설명
`QUETTA_GATEWAY_URL`	`http://localhost:8701`	Gateway API 주소
`QUETTA_ORCHESTRATOR_URL`	`http://localhost:8700`	Orchestrator 주소
`QUETTA_API_KEY`	*(없음)*	외부 접근 시 필요
`QUETTA_TIMEOUT`	`300`	요청 타임아웃 (초)
`QUETTA_REMOTE_AGENT_ID`	*(없음)*	기본 원격 에이전트 ID
`GEMINI_CLI`	`gemini`	Gemini CLI 실행 파일 경로
`GEMINI_MODEL`	`gemini-2.5-pro`	사용할 Gemini 모델

Gemini CLI 설치 (선택)

논문·설계도 분석 시 시각 분석 품질 향상:

```
npm i -g @google/gemini-cli
gemini  # 첫 실행 시 Google OAuth (브라우저 열림)
```

- 무료 쿼터: 1000 requests/day
- 별도 API 키 불필요 (OAuth 캐시 사용)
- 미설치 시 자동 건너뜀 (파이프라인은 정상 동작)

업데이트

Claude 채팅에서

```
quetta_version   # 현재 버전 확인
quetta_update    # 자동 업데이트
```

→ 업데이트 후 **Claude Code 재시작** 필요.

터미널에서

```
curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/update.sh | bash
```

제거

```
claude mcp remove quetta-agents
```

10. 트러블슈팅

사용자가 자주 마주치는 문제와 해결 방법.

설치 관련

"claude mcp list에 quetta-agents가 안 나옴"

```
# 1. 기존 설정 제거 후 재설치
claude mcp remove quetta-agents
curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh | bash

# 2. 설치 확인
claude mcp list | grep quetta

# 3. Claude Code 완전 재시작
```

"401 Unauthorized"

`QUETTA_API_KEY` 값이 잘못됐거나 누락됐습니다. 서비스 관리자에게 올바른 키를 요청하세요.

원격 제어 관련

"원격 에이전트 미연결"

```
# 연결 상태 확인
quetta_remote_connect(action="list")

# 없으면 설치 링크 받기
quetta_remote_connect(action="install-link", os="windows")
```

"WebSocket 연결 끊김 → 자동 재연결"

에이전트는 자동으로 재연결합니다 (3초 → 60초 지수 백오프). 5~10초 후 다시 명령을 시도하세요.

"화면 제어 불가"

→ `pyautogui` 미설치 가능성. 원격 PC에서:

```
import pyautogui
print(pyautogui.size())
```

논문 분석 관련

"GPU 에이전트가 없음"

Nougat OCR은 GPU가 필요합니다. Claude 채팅에서:

```
/remote-agent
```

→ GPU PC에 에이전트 설치 후 재시도.

"Nougat 실행 실패"

```
# 수동 설치 확인
quetta_remote_shell(command="pip show nougat-ocr")

# 재설치
quetta_analyze_paper(file_path="...", install_nougat=True)
```

"Gemini 건너뛰어짐"

```
# Gemini CLI 설치 및 OAuth 로그인
npm i -g @google/gemini-cli
gemini
```

설계도 분석 관련

"지원하지 않는 형식"

PDF / PNG / JPG만 지원됩니다. DWG는 사전 변환 필요:

```
# 예시 도구
libreoffice --headless --convert-to png drawing.dwg
```

"PDF 텍스트 없음 (이미지 도면)"

스캔본 PDF인 경우 정상입니다. Gemini Vision만으로 분석 진행됩니다.

업로드 관련

"upload_list 실패"

설정이 불완전할 수 있습니다. 재설치를 권장합니다:

```
claude mcp remove quetta-agents
QUETTA_GATEWAY_URL=https://rag.quetta-soft.com \
QUETTA_API_KEY=발급받은_키 \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)
```

install.sh 가 내부 설정을 자동으로 완성해줍니다.

"업로드 후 file_id 분실"

```
quetta_upload_list()   # 전체 업로드 목록 조회
```

RAG 질의 관련

"인제스트된 논문/설계도가 없음"

```
# 확인
quetta_paper_query(list=True)
quetta_blueprint_query(list=True)

# 없으면 먼저 분석
quetta_analyze_paper(file_path="...")
quetta_analyze_blueprint(file_path="...")
```

"관련 내용을 찾지 못함"

- 질문을 좀 더 구체적으로
- `top_k` 값 증가 (기본 8 → 15)
- `filename` 필터로 특정 파일 지정

일반

"Claude Code 재시작 후 도구가 안 보임"

1. `claude mcp list` 로 등록 상태 확인
2. Claude Code 완전 종료 후 재실행 (단순 새 창 열기 아님)
3. 그래도 안 되면: `bash claude mcp remove quetta-agents # 재설치`

"버전 업데이트 후에도 옛날 도구만 보임"

```
quetta_version     # 현재 버전 확인
quetta_update      # 최신으로 업데이트
```

→ **Claude Code 재시작 필수.**

그 외 문제

GitHub Issues로 제보: https://github.com/choyunsung/quetta-agents-mcp/issues

로그 첨부 시 개인정보·민감 정보는 마스킹해주세요.

11. 공유 메모리 & 멀티 계정

Quetta MCP는 여러 Claude Code 계정이 **동일한 서버의 RAG 지식베이스**를 공유해 기억을 이어가도록 설계되어 있습니다.

핵심 개념

```
[계정 A] ──┐
[계정 B] ──┼──▶ [공유 Quetta Server] ──▶ [공유 RAG 지식베이스]
[계정 C] ──┘
```

- 같은 `QUETTA_GATEWAY_URL` + (개인 혹은 공유) API 키를 사용하면 모든 계정이 같은 지식베이스에 접근
- 한 계정에서 저장한 기억은 다른 계정에서 즉시 검색·인용 가능
- 새 세션 시작 시 자동으로 기억이 컨텍스트에 주입됨

3가지 기억 레이어

1. 자동 Q&A 저장

- `quetta_ask` / `quetta_auto` 호출 시 Q&A가 자동으로 RAG에 저장됨

- 사용자가 별도 조치할 필요 없음
- source 태그: `quetta-gateway`

2. 자동 컨텍스트 주입

- 질문할 때마다 Gateway의 RAG harness가 관련 기억을 자동 검색해 LLM 입력에 삽입
- score ≥ 0.35 인 top 4개 청크를 system 메시지로 주입
- 질문·답변이 쌓일수록 컨텍스트 품질 향상

3. 명시적 사용자 메모리

도구	설명
`quetta_memory_save(text, tags?)`	고정 사실·선호·결정사항 영구 저장
`quetta_memory_recall(query)`	의미 검색 (LLM 답변 없이)
`quetta_memory_list()`	내가 저장한 메모리 목록
`quetta_session_init()`	세션 시작 시 컨텍스트 프리앰블 생성

세션 자동 초기화

install.sh 가 `~/.claude/CLAUDE.md` 에 다음 지시를 **자동으로** 추가합니다:

```
<!-- quetta-agents-mcp:auto-init BEGIN -->
## Quetta Agents MCP - 공유 메모리 자동 초기화

새 Claude Code 세션 시작 직후에 아래 도구를 한 번 호출하세요:
    quetta_session_init()

반환된 사용자 메모리 / 최근 맥락 / 활성 문서를 현재 대화의 배경 지식으로 반영합니다.
- 중요한 신규 정보는 quetta_memory_save(text=...) 로 공유 메모리에 저장하세요.
- 모든 quetta_ask / quetta_auto 호출은 Gateway RAG harness가 관련 메모리를 자동 주입합니다.
<!-- quetta-agents-mcp:auto-init END -->
```

→ Claude Code가 세션 시작 시 이 지시를 읽고 자동으로 공유 메모리 로드.

사용 예시

계정 A: 기억 저장

```
quetta_memory_save(
    text="사용자는 MCG 연구자. KRISS 96채널 SQUID 시스템 사용. Python 3.11 + FastAPI 선호.",
    tags=["profile", "mcg"],
)
# 또는 자연어로:
quetta_auto(request="기억해줘: 이 프로젝트는 Python 3.11 + FastAPI 기반이다")
```

계정 B: 자동 복원

Claude Code 새 세션 → `quetta_session_init()` 자동 호출 →

```
## 🎯 Quetta 세션 초기화 (공유 메모리 로드)

### 👤 사용자 메모리
- 사용자는 MCG 연구자. KRISS 96채널 SQUID 시스템 사용. Python 3.11 + FastAPI 선호. _[profile, mcg]_

### 💬 최근 대화 맥락
- Q: ECG ischemia 마커 AUC?  A: clin_score 0.929, ensemble_score 0.832...

### 📚 활성 문서
- [paper] attention.pdf
- [blueprint] cpld_top.pdf
```

이제 계정 B에서 별도 설명 없이 이어서 작업 가능.

자연어 단축 명령 (quetta_auto)

요청	자동 라우팅
"기억해줘: X"	`quetta_memory_save(text="X")`
"전에 뭐였지?"	`quetta_memory_recall`
"내 기억 목록"	`quetta_memory_list`
"저장된 논문에서 찾아"	`quetta_paper_query`

프라이버시 / 공유 범위

- **같은 API 키 사용 = 같은 지식베이스 공유**
- 개인 데이터가 팀에 공유되지 않기를 원하면 **개인 API 키** 발급 요청
- `quetta_memory_save` 시 `source` 파라미터로 네임스페이스 분리 가능

관련 문서

- 도구 레퍼런스
- 설치 & 설정
- 초대 토큰 설치 시스템

12. 비공개 설치 시스템 (2가지 방식)

Quetta MCP는 아무나 설치할 수 없습니다. 설치 권한을 배포하는 2가지 방식을 지원합니다.

방식 A: GitHub Secret Gist (권장)

관리자가 GitHub에 **secret Gist**로 설정값을 올리고, Gist ID를 팀원에게 전달.

팀원 설치

```
# 방법 1: gh CLI 인증 사용 (자동)
gh auth login  # 한 번만
QUETTA_GIST_ID="de6ac976511b92dd39115a0ac39626ff" \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)

# 방법 2: GitHub PAT 사용
GH_TOKEN=ghp_xxx QUETTA_GIST_ID="de6ac976..." \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)
```

관리자가 Gist 생성

```
# 1. config.json 준비
cat > quetta-install-config.json <<EOF
{
  "gateway_url": "https://rag.quetta-soft.com",
  "orchestrator_url": "https://rag.quetta-soft.com/orchestrator",
  "api_key": "YOUR_API_KEY",
  "rag_url": "https://rag.quetta-soft.com",
  "tusd_url": "https://rag.quetta-soft.com",
  "tusd_token": "YOUR_TUSD_TOKEN",
  "rag_key": "rag-claude-key-2026",
  "timeout": "300"
}
EOF

# 2. Secret Gist 생성 (secret = default)
gh gist create quetta-install-config.json -d "Quetta MCP install config"
# → https://gist.github.com/USER/<GIST_ID>

# 3. 팀원에게 Gist ID 전달
# 팀원은 gh CLI로 해당 Gist 읽기 권한이 있어야 함 (비공개 Gist는 소유자 + 공유받은 사람만)
```

장점

- 관리가 GitHub UI에서 이루어짐 — 편집, 취소, 권한 관리
- 팀원 인증이 GitHub 계정으로 통합됨
- 설정 파일 버전 관리 (Gist revisions)

주의

- "Secret Gist"는 **URL을 아는 누구나 접근 가능**. 진짜 비공개는 Gist를 만든 계정만 접근 가능한 건 아님 — URL 추측 불가.
- 완전 비공개가 필요하면 private GitHub repo + team 권한 사용

방식 B: Gateway 초대 토큰 (GitHub 불필요)

관리자가 Gateway에서 직접 토큰 발급 → 팀원에게 전달.

팀원 설치

```
QUETTA_INSTALL_TOKEN="<받은_토큰>" \
bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)
```

install.sh 동작: 1. Gateway `GET /install/config?token=<토큰>` 호출 2. 토큰이 유효하면 설정값(API 키, URL 등) 수신 3. `claude mcp add-json` 으로 Claude Code에 등록 4. `~/.claude/CLAUDE.md` 에 세션 자동 초기화 지시 추가 5. **끝** — 사용자는 API 키를 직접 볼 필요도, 관리할 필요도 없음

토큰 미지정 시 대화형 입력 프롬프트가 나옵니다.

관리자 흐름 (토큰 발급)

1. 토큰 생성

```
# 서버에서 (또는 API_KEY 있는 곳에서)
/data/quetta-agents/scripts/invite.sh create "팀원A"
```

출력:

```
✅ 초대 토큰 발급 완료 (팀원A, 365일)

┌──────────────────────────────────────────────────────────────────────────────┐
│ 아래 명령어를 팀원에게 전달하세요:                                          │
└──────────────────────────────────────────────────────────────────────────────┘

 QUETTA_INSTALL_TOKEN="KjX9sQ_pLm..." \
 bash <(curl -fsSL https://raw.githubusercontent.com/choyunsung/quetta-agents-mcp/master/install.sh)
```

이 명령어를 Slack / 메일 / DM으로 팀원에게 전달합니다.

2. 유효기간 지정

```
# 7일만 유효
/data/quetta-agents/scripts/invite.sh create "임시-테스터" 7

# 무기한 (0)
/data/quetta-agents/scripts/invite.sh create "영구-사용자" 0
```

3. 목록 조회

```
/data/quetta-agents/scripts/invite.sh list
```

출력:

```
라벨                 prefix          uses   상태    만료
--------------------------------------------------------------------------------
팀원A               KjX9sQ_pLm      3     ✓active  2027-04-13
임시-테스터          9mNxYp0qRt      1     ✓active  2026-04-20
영구-사용자          Ab3Cd5Ef7g      12    ✓active  ∞
전직_인턴            Zx2Yw4Vu6t      8     ❌revoked 2026-10-10
```

4. 토큰 취소

```
# prefix 12자로 지정 (충돌 방지)
/data/quetta-agents/scripts/invite.sh revoke KjX9sQ_pLm
```

취소된 토큰은 **즉시 무효화** — 이미 설치된 사용자도 다음 요청부터 실패합니다(설치된 MCP의 실제 API 키가 따로 유효하면 계속 사용 가능, 영구 차단은 master 키 교체).

REST API (직접 호출)

토큰 발급 (admin)

```
curl -X POST https://rag.quetta-soft.com/install/invite \
  -H "Authorization: Bearer <MASTER_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"label":"팀원A","expires_days":365}'
```

토큰으로 config 조회 (public — 인증 불필요)

```
curl https://rag.quetta-soft.com/install/config?token=<INVITE_TOKEN>
```

토큰 목록 (admin)

```
curl https://rag.quetta-soft.com/install/invites \
  -H "Authorization: Bearer <MASTER_API_KEY>"
```

토큰 취소 (admin)

```
curl -X DELETE https://rag.quetta-soft.com/install/invite/<PREFIX> \
  -H "Authorization: Bearer <MASTER_API_KEY>"
```

보안 고려사항

항목	설명
토큰 저장	Gateway 호스트 `/data/quetta-agents/storage/install_tokens.json`
토큰 형식	`secrets.token_urlsafe(24)` — 32자, URL-safe
토큰 검증	O(1) lookup (JSON memory + disk persistence)
만료	발급 시 지정 (기본 365일)
사용 기록	`uses`, `last_used` 필드에 기록
Rate limit	nginx-level 권장 (`/install/config` 엔드포인트)
토큰 유출 시	`revoke` 또는 master API 키 회전

토큰 vs 직접 API 키

방식	용도	장점	단점
초대 토큰	팀원 배포	키 노출 없음, 취소 가능, 사용 추적	Gateway 접근 필요
직접 API 키 (`QUETTA_API_KEY`)	서버-to-서버	완전 오프라인 사용 가능	키 관리 필요

일반 사용자에게는 **초대 토큰**을 권장합니다. 자동화/스크립트에는 직접 API 키.

관련 문서

- 설치 & 설정
- 공유 메모리

13. 대화 히스토리 (NoSQL)

모든 `quetta_ask` / `quetta_auto` 호출이 자동으로 **MongoDB에 저장**됩니다. 계정별 누적 + 통합 조회 모두 지원.

아키텍처

```
Claude Code → quetta_ask / quetta_auto
              ↓
     Quetta Gateway /v1/chat/completions
             ├── RAG harness (context injection)
             ├── LLM 응답 생성
             ├── save_qa  → RAG vector DB (검색용)
             └── save_conversation → MongoDB (이력용) ← NEW
                                     │
                                     ▼
                           quetta_agents.conversations
                           (계정별 user_hash로 구분)
```

저장 데이터 구조

`quetta_agents.conversations` 컬렉션:

```
{
  _id: ObjectId,
  session_id: "default",         // X-Session-Id 헤더 기반
  timestamp: 1776062400.123,
  user_hash: "a3f5b2c1...",      // SHA-256(API_KEY)[:16] - 계정 식별
  query: "사용자 질문",
  response: "LLM 답변",
  backend: "claude-sonnet",
  model: "claude-sonnet-4-6",
  rag_chunks: 4,
  tokens: { input: 150, output: 320 },
  routing: { reason, is_medical, inject_code_skills, multi_agent }
}
```

인덱스: - `(session_id, timestamp)` — 세션별 시간순 - `(user_hash, timestamp DESC)` — 사용자별 최근순 - `(timestamp DESC)` — 전체 최근순

개인정보 보호

- API 키는 **SHA-256 해시**로만 저장 (역추적 불가)
- 같은 API 키 → 같은 `user_hash` → 동일 사용자로 그룹핑
- 다른 키 사용 시 다른 사용자로 분리

MCP 도구

`quetta_history_list`

파라미터	기본	설명
`mine_only`	true	내 API 키 세션만
`unified`	false	true면 모든 사용자 통합
`limit`	30	최대 반환 세션 수

`quetta_history_get`

파라미터	설명
`session_id`	조회할 세션 ID (필수)
`limit`	기본 100

`quetta_history_stats`

파라미터 없음 — 전체 통계 요약.

사용 예시

```
# 내 대화 세션 목록
quetta_history_list()
# → ## 💬 대화 히스토리 (내 계정, 12개 세션)
#    - `default`  (47개 메시지, 04-10 09:22~04-13 08:51)
#      "ECG ischemia 마커..."
#    - `project-mcg`  (31개 메시지, ...)

# 전체 사용자 통합 조회
quetta_history_list(unified=True, limit=50)

# 특정 세션 상세
quetta_history_get(session_id="project-mcg")
# → ## 🗂 세션 project-mcg (31개 메시지)
#    ### 2026-04-12 15:30:00 [claude-sonnet]
#    Q: ...
#    A: ...

# 전체 통계
quetta_history_stats()
# → 총 대화 수: 1,247
#    고유 세션 수: 23
#    고유 사용자 수: 4
#    백엔드별: claude-sonnet 842 / gemma4 305 / deepseek-r1 100
```

세션 ID 지정

기본 세션 ID는 `default` 입니다. 특정 주제별로 분리하려면 HTTP 요청에 `X-Session-Id` 헤더 포함:

```
# MCP 차원에서 헤더 지정은 아직 미지원 — gateway 직접 호출 시만 가능
# 향후 quetta_ask 에 session_id 파라미터 추가 예정
```

계정별 vs 통합 조회

시나리오	호출
내가 전에 뭘 물어봤지?	`quetta_history_list()` (mine_only=True, 기본)
팀 전체 대화 흐름	`quetta_history_list(unified=True)`
특정 세션 되돌아보기	`quetta_history_get(session_id="X")`
전체 활동 요약	`quetta_history_stats()`

관리자 기능

세션 삭제

```
curl -X DELETE https://rag.quetta-soft.com/history/session/<SID> \
  -H "Authorization: Bearer <MASTER_API_KEY>"
```

REST API

Method	Path	Auth
GET	`/history/sessions?limit=30&user_hash=X`	API key
GET	`/history/session/{sid}?limit=200`	API key
DELETE	`/history/session/{sid}`	Master key
GET	`/history/stats`	API key

데이터 보관

- **백업 대상**: MongoDB `quetta_agents.conversations` 컬렉션
- **용량 예상**: 대화 1건 ≈ 5KB → 10만건 ≈ 500MB
- **정리 정책**: 수동 (기본은 무기한 보관). 필요 시 TTL 인덱스로 자동 삭제 가능
- **GDPR/개인정보**: `user_hash` 는 해시이므로 API 키 자체가 삭제되면 재매칭 불가

관련 문서

- 공유 메모리
- 초대 토큰 설치
- 도구 레퍼런스

14. 워크스페이스 (멀티 테넌트)

업무용 / 개발용 지식을 분리해서 사용자별로 접근 권한을 제어하는 기능.

핵심 개념

```
[Admin (master API key)]  ── 모든 워크스페이스 접근
     │
     │ ACL 관리
     ▼
[Workspace: development]  ← 기본 생성
[Workspace: business]     ← 기본 생성
[Workspace: project-X]    ← 관리자가 추가 가능
     │
     ▼
[사용자 A (api_key A)]  ← 관리자가 ["development"] 만 부여
[사용자 B (api_key B)]  ← 관리자가 ["business"] 만 부여
[사용자 C (api_key C)]  ← 관리자가 ["development", "business"] 둘 다 부여
```

왜 필요한가

- **비개발자 사용자**는 개발 관련 RAG 내용이 섞이면 혼란스러움
- **업무 기밀**은 개발자에게 불필요하고, 보안상 분리 필요
- 한 RAG 인스턴스에서 **네임스페이스로 격리**하면서도 **관리자는 전체 통합** 뷰 보유

자동 동작

호출	워크스페이스 동작
`quetta_ask(...)`	사용자 허용 ws에서만 RAG 검색 (harness)
`quetta_ask(...)` 저장	사용자 기본 ws에 저장 (`X-Workspace` 헤더로 override 가능)
`quetta_memory_save(text="...")`	`workspace` 파라미터 또는 기본값
Admin 호출	전체 ws 조회/저장 가능

기본 워크스페이스 (Gateway 최초 실행 시 생성)

이름	레이블	설명
`development`	개발 지식	코드, 아키텍처, 기술 스택, 트러블슈팅 (기본)
`business`	업무 지식	프로젝트 결정사항, 회의록, 일정

사용자 도구

`quetta_workspace_list`

```
quetta_workspace_list()
# → ## 내 워크스페이스 정보
#    - user_hash: abc1234567890def
#    - 관리자: ❌
#    - 기본 워크스페이스: development
#    - 접근 가능: ['development']
#
#    ### 전체 워크스페이스
#    - ✅ **development** (기본) - 개발 지식: 코드, 아키텍처, ...
#    - 🔒 **business** - 업무 지식: 프로젝트 결정사항, ...
```

`quetta_workspace_request`

```
quetta_workspace_request(workspace="business", reason="프로젝트 A 회의 참여")
# → ⏳ 요청 접수됨 - 관리자 승인 대기 중
```

`quetta_memory_save` (workspace 지정)

```
# 업무 메모 (해당 워크스페이스 권한 필요)
quetta_memory_save(
    text="2026-04-15 프로젝트 A 킥오프 미팅: 일정 결정됨 ...",
    workspace="business",
    tags=["meeting", "project-A"],
)

# 개발 메모 (기본)
quetta_memory_save(
    text="Gateway 502 원인: Ollama 바인딩 127.0.0.1 → 0.0.0.0",
    workspace="development",
    tags=["troubleshooting"],
)
```

관리자 도구 (master API key 필수)

`quetta_admin_requests`

```
quetta_admin_requests()
# → ## ⏳ 대기 중인 접근 요청 (2건)
#    - `abc1234567890def` → **business** (2026-04-13 15:30)
#      이유: 프로젝트 A 회의 참여
```

`quetta_admin_resolve` (승인/거부)

```
quetta_admin_resolve(
    user_hash="abc1234567890def",
    workspace="business",
    approve=True,
)
# → ✅ 승인됨
```

`quetta_admin_grant` (일괄 ACL 설정)

```
# 특정 사용자의 ACL 전체 교체
quetta_admin_grant(
    user_hash="abc1234567890def",
    workspaces=["development", "business"],  # 전체 허용
)
```

`quetta_admin_create_workspace` (새 워크스페이스)

```
quetta_admin_create_workspace(
    name="project-healthcare",
    label="헬스케어 프로젝트",
    description="의료 프로젝트 전용 지식베이스",
)
```

흐름 예시

시나리오 1: 비개발자 입사

1. Admin: `quetta_admin_create_workspace(name="ops", label="운영")`
2. 신입: Gist로 설치 → `quetta_workspace_request(workspace="ops")`
3. Admin: `quetta_admin_requests()` → 확인
4. Admin: `quetta_admin_resolve(user_hash="X", workspace="ops", approve=True)`
5. 이후 신입의 모든 `quetta_ask` 는 `ops` 워크스페이스에서만 검색/저장

시나리오 2: 권한 분리

- 개발자: `development` 만 부여
- 영업: `business` 만 부여
- PM: 두 워크스페이스 모두 부여

각자 quetta_ask 하면 자기 권한 내 데이터만 보임.

시나리오 3: Admin이 통합 조회

master API key 사용자는 `quetta_ask` 시 **전체** 워크스페이스에서 검색: - `X-Workspace: development` 헤더 주면 해당 ws만 한정 - 헤더 없으면 전체 통합

저장 메타데이터

모든 RAG 청크에 `workspace` 필드 자동 추가:

```
{
  "text": "...",
  "source": "quetta-gateway:business",
  "metadata": {
    "workspace": "business",
    "backend": "claude-sonnet",
    ...
  }
}
```

REST API

Method	Path	권한
GET	`/workspace/me`	누구나 (자기 정보)
POST	`/workspace/request`	누구나 (자기 요청)
POST	`/workspace/create`	Admin
DELETE	`/workspace/{name}`	Admin
POST	`/workspace/acl/set`	Admin
GET	`/workspace/acl`	Admin
GET	`/workspace/requests?status=pending`	Admin
POST	`/workspace/resolve`	Admin

관련 문서

- 공유 메모리
- 설치 시스템
- 대화 히스토리